ПОРТФОЛИО · ТЕОРИЯ И ПРАКТИКА

QA: вопросы и ответы

Денис Курчиков, QA-инженер

59	9	1
вопросов с ответами	тематических разделов	задача на код с автотестами

О документе

Подборка вопросов из технических тестов, которые я проходил при отклике на вакансии ручного тестирования, автотестирования и SDET. Вопросы сгруппированы по темам, к каждому дан ответ и коротко объяснена логика: почему этот вариант верный и чем он отличается от похожих.

Документ показывает, какие темы я знаю и как рассуждаю: от основ тестирования и тест-дизайна до сети, мобильных платформ, автоматизации и CI/CD.

Разделы

- **1. Основы тестирования** · 11
- **2. Тест-дизайн** · 2
- **3. Баги и тестовая документация** · 5
- **4. Веб, сеть и браузер** · 16
- **5. Мобильное и кроссбраузерное тестирование** · 4
- **6. API** · 1
- **7. Коммуникация с разработкой и клиентом** · 3
- **8. Автоматизация и CI/CD** · 14
- **9. Python** · 3
- **Практика: задача на код с автотестами**

РАЗДЕЛ 1

Основы тестирования

Виды тестирования, принципы, жизненный цикл, severity и priority.

1. Регрессионное тестирование нужно, чтобы проверить...

> **Ответ:** что после последних изменений не появились новые дефекты и ранее работавшая функциональность работает как прежде.

Проверка одного конкретного фикса - это подтверждающее (confirmation) тестирование, проверка новой фичи - функциональное. Регрессия про то, что изменения ничего не сломали вокруг.

2. Как называется тестирование без документации и заранее написанных тест-кейсов?

> **Ответ:** Исследовательское (exploratory) тестирование.

Тестировщик одновременно изучает продукт, придумывает сценарии и сразу их выполняет, опираясь на опыт.

3. Тестировщик карандаша решил его пожевать, сунуть в розетку, намочить и высушить. Какой это вид тестирования?

> **Ответ:** Негативное тестирование.

Все сценарии - использование объекта не по назначению, чтобы посмотреть, как он себя поведёт и где сломается.

4. Как называется самая ранняя версия сборки для внутреннего тестирования?

> **Ответ:** Альфа.

Альфа тестируется внутри команды, бета - ограниченным кругом внешних пользователей, релиз выходит для всех. Debug-сборка - это не стадия релизного цикла, а тип сборки с отладочной информацией.

5. Что такое предусловие в тест-кейсе?

> **Ответ:** Подготовительные действия, без которых кейс нельзя выполнить: настройка окружения, регистрация пользователя, нужные данные.

6. Что из перечисленного не входит в жизненный цикл тестирования (STLC): планирование, улучшение ПО, анализ результатов, закрытие, исправление багов?

> **Ответ:** Улучшение ПО и исправление багов.

Это задачи разработки. Этапы STLC: анализ требований, планирование, разработка тест-кейсов, подготовка окружения, выполнение тестов, закрытие. Анализ результатов отдельным этапом не выделяется, но входит в выполнение и закрытие.

7. Какое определение лучше всего описывает парадокс пестицида?

> **Ответ:** Тесты со временем становятся менее эффективными.

Если гонять одни и те же тесты, они перестают находить новые баги, поэтому их нужно регулярно пересматривать. Важно не путать с принципом скопления дефектов: большинство багов сосредоточено в небольшом числе модулей.

8. Новый билд только что задеплоен на staging. Что QA делает в первую очередь?

> **Ответ:** Быстрый smoke-тест ключевых сценариев.

Сначала убеждаемся, что сборка в принципе работоспособна, и только потом идём в глубину. Полная регрессия на первом шаге слишком долгая.

9. Разработчик исправил баг на странице checkout. Что проверить перед релизом?

> **Ответ:** Весь флоу оформления заказа и связанную функциональность.

Мало проверить саму исправленную кнопку: нужно убедиться, что фикс не сломал соседние шаги. Это сочетание подтверждающего и регрессионного тестирования.

10. В шапке сайта опечатка в названии компании, видна на каждой странице. Как её классифицировать?

> **Ответ:** Low severity, high priority.

Функциональность не страдает, поэтому серьёзность низкая. Но опечатку видят все и на каждой странице, это репутационный риск, поэтому чинить нужно срочно.

11. За час до релиза найдены баги: краш при открытии экрана, потеря данных после сохранения формы, неверный текст подсказки, загрузка 5 секунд при нормативе 2, неверный цвет кнопки. Какие из них блокеры?

> **Ответ:** Краш и потеря данных.

Блокер делает продукт нерабочим или необратимо вредит пользователю. Медленная загрузка серьёзна, но страница работает. Текст и цвет - косметика.

РАЗДЕЛ 2

Тест-дизайн

Техники, которые позволяют покрыть больше при меньшем числе проверок.

12. Какой метод тест-дизайна сокращает количество тестовых комбинаций?

Ответ: Попарное (pairwise) тестирование.

Граничные значения и классы эквивалентности сокращают число проверок внутри одного параметра. Попарное тестирование работает с комбинациями нескольких параметров: покрывает все пары значений без полного перебора.

13. При переводе 10 000 ₽ комиссии нет, при 10 001 ₽ она составляет 1%. Какая техника здесь подходит лучше всего?

Ответ: Анализ граничных значений.

Есть чёткая числовая граница, поэтому проверяем значения вокруг неё: 9 999, 10 000, 10 001, 10 002. Таблица решений нужна, когда комбинируется несколько условий.

РАЗДЕЛ 3

Баги и тестовая документация

Баг-репорты, выбор вида документации, работа с плавающими багами.

14. Какой заголовок бага лучший?

Ответ: «Кнопка “Place Order” не реагирует в Safari 17 / macOS на шаге checkout».

Из заголовка сразу понятно, что сломано, где и в каком окружении. Эмоциональные заголовки вроде «Ничего не работает!!!» бесполезны.

15. Что обязательно должно быть в баг-репорте?

Ответ: Шаги воспроизведения, ожидаемый результат, фактический результат, окружение (браузер, устройство, URL).

Догадка о том, какая строка кода сломана, в репорт не входит: QA описывает наблюдаемое поведение, а предположения могут увести разработчика по ложному следу.

16. Нужно быстро проверить сайт после небольшого обновления плагина WordPress. Что подходит лучше всего?

Ответ: Короткий чек-лист ключевых страниц и сценариев.

Полные тест-кейсы и регрессия здесь избыточны, а проверка только экрана настроек плагина слишком узкая: плагин может повлиять на весь сайт.

17. Дефект появляется только при определённой последовательности действий и не всегда. В каком порядке действовать?

Ответ: Повторить несколько раз, меняя условия → собрать логи, скриншоты, версии → попросить коллегу воспроизвести → проверить, не заведён ли баг раньше → завести баг-репорт.

Из предложенных вариантов этот самый логичный: репорт заводится, когда картина ясна. На практике я бы проверил дубликаты в самом начале, чтобы не тратить время на уже известный баг.

18. В мобильном приложении у части пользователей иногда не отправляются сообщения в чате, а логи на клиенте и сервере чистые. Что эффективнее всего для диагностики?

> **Ответ:** Повысить уровень логирования на сервере, проверить разные типы сети и потерю пакетов, поискать связь с моделями устройств и версиями ОС, проверить таймауты запросов.

Раз логи молчат, нужно больше данных. Чат сильно зависит от сети, фрагментация устройств часто объясняет баги «у части пользователей», а таймаут может не логироваться как ошибка. Расспросы пользователей - слабый первый шаг: технические условия они описать не смогут.

РАЗДЕЛ 4

Веб, сеть и браузер

Как устроены веб-приложения, протоколы, DevTools и типичная диагностика.

19. Кнопка выглядит нормально, но по клику ничего не происходит. Какой слой скорее всего сломан?

> **Ответ:** JavaScript.

HTML отвечает за структуру (кнопка есть), CSS за внешний вид (выглядит нормально), а поведение по клику - это JavaScript.

20. Кнопка формы не реагирует. В Console ошибка «Uncaught TypeError», в Network запрос не отправлялся. Что это?

> **Ответ:** Баг фронтенда (JavaScript).

Запрос даже не ушёл на сервер, значит проблема возникла раньше, в обработчике клика, и ошибка в Console это подтверждает.

21. Вводите «abc» в поле email и сразу, до отправки формы, видите «Invalid email». Где происходит валидация?

> **Ответ:** Во фронтенде, в JavaScript.

Мгновенная реакция без сетевого запроса - признак проверки в браузере. Серверная валидация потребовала бы отправки данных.

22. Вы залогинились, закрыли браузер, а на следующий день всё ещё авторизованы. За счёт чего?

> **Ответ:** Сессия хранится в cookie или local storage.

23. Что такое cookie и кэш браузера?

> **Ответ:** Cookie - небольшие данные о пользователе и его визитах, которые сайт хранит в браузере. Кэш - временное хранилище уже скачанных ресурсов (картинок, скриптов, стилей).

24. Картинка товара не загружается. Где в DevTools посмотреть упавший запрос и его статус-код?

> **Ответ:** Во вкладке Network.

Elements - это DOM, Application - хранилища, Sources - исходный код и отладка JS.

25. Как в DevTools эмулировать медленный интернет?

> **Ответ:** Network → выпадающий список «No throttling» → выбрать профиль медленного соединения.

Network Link Conditioner - это отдельный инструмент macOS и iOS, к DevTools браузера он не относится.

26. Ссылка в футере показывает «Page not found». Какой статус-код будет в Network?

> **Ответ:** 404.

200 - успешный ответ, 301 - постоянное перенаправление, 500 - внутренняя ошибка сервера.

27. Какой протокол используется для безопасного соединения с сайтами?

Ответ: HTTPS.

Это HTTP поверх шифрования TLS. FTP служит для передачи файлов, TCP - транспортный протокол более низкого уровня.

28. Какой протокол используется для удалённого доступа к серверу?

Ответ: SSH.

HTTP - для веба, FTP - для файлов, SMTP - для почты.

29. Что произойдёт, если DNS-сервер недоступен?

Ответ: Сайты перестанут открываться по доменным именам, но по IP-адресу подключиться можно.

DNS переводит имена в IP-адреса. Интернет при этом не пропадает, а кэш DNS живёт ограниченное время.

30. Что делает команда ping?

Ответ: Проверяет, отвечает ли удалённый узел, и замеряет время отклика.

31. Что такое IP-адрес?

Ответ: Числовой идентификатор устройства в сети.

Не путать с MAC-адресом - физическим адресом сетевой карты.

32. Как называется ссылка, которая открывает конкретный раздел приложения или сайта?

Ответ: Deep link (диплинк).

В отличие от обычной ссылки, ведёт не на главный экран, а вглубь приложения.

33. Что такое саджест?

Ответ: Выпадающий список подсказок при вводе, например вариантов поискового запроса.

34. Какой CSS-селектор выберет абзацы, идущие непосредственно после изображений внутри статьи?

Ответ: article img + p

«+» выбирает соседа, идущего сразу следом. Пробел - потомок на любом уровне, «>» - прямой дочерний элемент, «~» - любой последующий сосед, не обязательно соседний.

РАЗДЕЛ 5

Мобильное и кроссбраузерное тестирование

Особенности мобильных платформ, браузеров и анализа трафика.

35. Что проверить в мобильной версии интернет-магазина?

Ответ: Вёрстку на разных ширинах экрана, размер элементов для тапа, портретную и альбомную ориентацию, поведение при открытой экранной клавиатуре.

Проверять только на своём телефоне - неправильный подход: баги часто специфичны для других экранов и устройств.

36. Слайдер работает в Chrome, но сломан в Safari. Какой первый шаг?

Ответ: Воспроизвести в Safari, зафиксировать точную версию браузера и ОС, посмотреть консоль и завести баг с деталями окружения.

37. Как установить .ipa-файл на iOS-устройство?

> **Ответ:** Через Xcode, через корпоративный профиль (Enterprise Distribution) и через Apple Configurator (раньше для этого использовали iTunes).

Просто отправить .ipa по AirDrop или открыть из «Файлов» нельзя: iOS не устанавливает такие файлы без дополнительных инструментов.

38. Приложение по HTTPS может передавать конфиденциальные данные на сторонний сервер. Charles Proxy настроен, телефон ходит через него, но трафика приложения не видно. Почему?

> **Ответ:** Приложение использует Certificate Pinning.

Приложение доверяет только конкретному сертификату сервера. Сниффер подменяет сертификат, приложение видит несовпадение и обрывает соединение, хотя остальной интернет на телефоне работает.

РАЗДЕЛ 6

API

39. Сложный вложенный GraphQL-запрос возвращает 500 Internal Server Error, доступа к серверным логам нет. Как найти причину?

> **Ответ:** В GraphQL Playground или Altair постепенно упрощать запрос, убирая поля и вложенности, пока ошибка не исчезнет.

Без логов это единственный способ локализовать проблемное поле. Повторный запрос лишь подтвердит воспроизводимость, а баг-репорт всё равно понадобится, но уже с найденной причиной.

РАЗДЕЛ 7

Коммуникация с разработкой и клиентом

Как собирать информацию и спокойно разбирать расхождения.

40. Клиент пишет: «Кнопка checkout не работает». Что делать в первую очередь?

> **Ответ:** Попробовать воспроизвести и запросить браузер, устройство, URL и скриншот или видео.

41. Проблема воспроизводится только у одного пользователя. Что собрать?

> **Ответ:** Браузер и версию, устройство и ОС, ID аккаунта и точные шаги, скриншот или запись экрана, воспроизводится ли после очистки кэша и в инкогнито.

Пароль пользователя не запрашиваем никогда: для диагностики он не нужен.

42. Разработчик говорит: «У меня всё работает». Что делает QA?

> **Ответ:** Воспроизводит ещё раз и делится с разработчиком точным окружением, шагами и коротким видео.

Не закрывать баг на слово, не спорить без доказательств и не эскалировать, не разобравшись. Точные данные быстрее всего показывают разницу в условиях.

РАЗДЕЛ 8

Автоматизация и CI/CD

Пирамида тестирования, стабильность автотестов, пайплайны.

43. Как определить приоритет автоматизации тестов?

Ответ: Автоматизировать в первую очередь сценарии, которые выполняются часто и одновременно сильно влияют на бизнес.

Частые, но некритичные сценарии - трата ресурсов, «легко автоматизировать» - не критерий ценности, а только UI E2E-тесты самые дорогие в поддержке.

44. Бюджет на автоматизацию небольшой. Какой уровень пирамиды в приоритете?

Ответ: Максимум unit-тестов, минимум UI-тестов.

Unit-тесты самые дешёвые, быстрые и стабильные: максимум пользы при минимуме затрат.

45. Как сократить время прогона автотестов, не теряя качества?

Ответ: Разложить тесты по уровням пирамиды: быстрые нижнего уровня гонять часто и полностью, медленные E2E - точечно, сохраняя критичное покрытие.

46. Пайплайн запускает все 500 тестов на каждый пуш, прогон идёт 40 минут. Как оптимизировать?

Ответ: На feature-ветках запускать только smoke, полный прогон - при мерже в main.

Быстрая обратная связь на каждый пуш и полная проверка перед попаданием в основную ветку. Разделить тесты на два пайплайна, которые идут друг за другом, ничего не даст.

47. За час до релиза менеджер просит «быстро протестировать» фикс. Что делать?

Ответ: Запустить быстрый регрессионный набор автотестов и явно сообщить, какие области остались непокрытыми.

Используем автоматизацию по максимуму и честно обозначаем риски, без ложного ощущения полной проверки.

48. Автотест проходит на DEV, но стабильно падает на STAGE, окружения «идентичны». Первые действия?

Ответ: Перезапустить тест на STAGE несколько раз, посмотреть логи сервера и браузера на STAGE, сравнить тестовые данные и конфигурацию DEV и STAGE.

Сначала исключаем случайность, потом ищем реальную разницу: «идентичные» окружения на практике редко такими бывают. Сразу писать баг на разработку без диагностики непрофессионально.

49. Автотест иногда проходит, иногда падает без изменений в коде. С чего начать?

Ответ: Добавить подробное логирование (ответы API, состояние БД, время) и проверить, не зависит ли тест от данных, оставленных другими тестами.

Общее состояние между тестами (shared state) - самая частая причина flaky-тестов. Отключить тест - значит спрятать проблему, а не разобраться.

50. После мержа несвязанной фичи CI падает: три разных теста в случайном порядке с ElementNotInteractableException. Первые три действия?

Ответ: Запустить падающие тесты локально, проверить изменения в репозитории тестов за последние сутки, проверить, не обновлялись ли браузер и веб-драйвер на CI.

Случайный порядок падений после несвязанного изменения указывает на инфраструктуру, например автообновление драйвера, а не на саму фичу.

51. В чём проблемы этого теста с точки зрения надёжности?

```
def test_login():
    driver.get("https://site.com/login")
    username_field = driver.find_element(By.ID, "username")
    password_field = driver.find_element(By.ID, "password")
    username_field.send_keys("admin")
    password_field.send_keys("password123")
    driver.find_element(By.ID, "login-btn").click()
    assert driver.current_url == "https://site.com/dashboard"
```

> **Ответ:** Нет явных ожиданий (Explicit Waits), а логин и пароль захардкожены в тесте.

После клика сразу идёт assert без ожидания перехода страницы - классическая причина нестабильности. Тестовые данные стоит выносить из кода теста. Локаторы по ID, наоборот, одни из самых надёжных.

52. Элемент появляется через несколько секунд после загрузки страницы. Как надёжнее всего с ним работать?

> **Ответ:** Использовать явное ожидание (Explicit Wait) условия появления элемента.

Thread.sleep - жёсткая пауза, неявное ожидание плохо сочетается с явным, а увеличение глобального таймаута замедляет все тесты.

53. Что такое моки?

> **Ответ:** Тестовые дубли, которые позволяют контролировать вызовы методов, переданные аргументы и проверять ожидаемое поведение тестируемой системы.

Не путать с другими дублями: fake - упрощённая рабочая реализация, spy - записывает информацию о своём использовании.

54. Что относится к мутациям в мутационном тестировании?

> **Ответ:** Замена переменных на другие из той же области видимости, изменение условий в циклах и ветвлениях, замена арифметических операторов.

Мутируется сам код, чтобы проверить, поймают ли это тесты. Изменение входных данных - это работа с тестовыми данными, а не мутация.

55. Чем Continuous Integration отличается от Continuous Delivery?

> **Ответ:** CI - частое слияние кода в общую ветку с автоматической сборкой и прогоном тестов. CD - расширение CI: любая сборка, прошедшая тесты, готова к выкладке в продакшен в любой момент.

56. Как QA эффективнее всего участвовать в разработке фичи?

> **Ответ:** Подключаться на ранних стадиях: предлагать тестовые сценарии, оценивать риски и влиять на качество требований.

Это подход shift-left: дешевле всего баги, которые не попали в код, потому что их заметили ещё на уровне требований.

РАЗДЕЛ 9

Python

Чтение кода и работа со списками.

57. Что выведет код?

```
def find_even_numbers(numbers):
    return [num for num in numbers if num % 2 == 0]

print(find_even_numbers([1, 2, 3, 4, 5, 6]))
```

Ответ: [2, 4, 6]

List comprehension оставляет только чётные элементы.

58. Что выведет код с .append?

```
def process_numbers(data):
    result = []
    for key, values in data.items():
        evens = [num for num in values if num % 2 == 0]
        if evens:
            result.append(evens)
    return result

data = {"a": [1, 2, 3], "b": [4, 5, 6], "c": [7, 8, 9]}
print(process_numbers(data))
```

Ответ: [[2], [4, 6], [8]]

.append() добавляет список evens целиком как один элемент, поэтому результат вложенный.

59. А если заменить .append на .extend?

Ответ: [2, 4, 6, 8]

.extend() добавляет элементы по одному, и список получается плоским.

ПРАКТИКА

Задача на код с автотестами

Условие: убрать из списка подряд идущие повторы и покрыть решение автотестами. Язык на выбор, решение на Python и pytest.

Идея решения: идём по списку и добавляем число в результат, только если оно отличается от последнего уже добавленного. Так подряд идущие повторы схлопываются, а несоседние повторы, например [1, 2, 1], остаются.

compress_numbers.py

```
def compress_numbers(numbers):
    """
    Removes consecutive duplicate numbers from a list.
    Example: [1, 1, 2, 2, 3] -> [1, 2, 3]
             [0, 0, 1, 1, 0] -> [0, 1, 0]
    """
    if not numbers:
        return []
    result = [numbers[0]]  # always keep the first element
    for num in numbers[1:]:
        # only add the number if it differs from the last one we kept
        if num != result[-1]:
            result.append(num)
    return result
```

test_compress_numbers.py

Тесты покрывают основной сценарий, примеры из условия, граничные случаи (пустой список, один элемент, все элементы одинаковые) и ключевое требование: несоседние повторы не удаляются.

```
from compress_numbers import compress_numbers

def test_basic_duplicates():
    # Consecutive duplicates in the middle should collapse
    assert compress_numbers([1, 1, 2, 2, 3]) == [1, 2, 3]

def test_example_from_task():
    # Second example from the task description
    assert compress_numbers([0, 0, 1, 1, 0]) == [0, 1, 0]

def test_empty_list():
    # Edge case: empty input should return empty output, not crash
    assert compress_numbers([]) == []

def test_single_element():
    # Edge case: one element, nothing to compress
    assert compress_numbers([5]) == [5]

def test_no_duplicates():
    # No consecutive duplicates - the list stays unchanged
    assert compress_numbers([1, 2, 3, 4]) == [1, 2, 3, 4]

def test_all_same_element():
    # Every element is the same - collapses to a single value
    assert compress_numbers([7, 7, 7, 7]) == [7]

def test_non_consecutive_duplicates_are_kept():
    # Only consecutive duplicates are removed, not all duplicates
    assert compress_numbers([1, 2, 1, 2]) == [1, 2, 1, 2]
```